VORTEXA 3.0 HACKATHON · PROBLEM STATEMENT E-02

# CodeQuest AI

## Solution Proposal & Architecture Report

*An AI coding mentor and debugging-mission platform: deterministic diagnosis, guarded hints, private one-to-one chat, personalised explanations and a trilingual (English, Hindi, Marathi) voice mentor.*

# Contents

**1. Executive Summary ..... 4**
1.1 What makes the solution different ..... 4
1.2 Headline success measures ..... 5
**2. Problem Statement and Opportunity ..... 6**
2.1 The problem ..... 6
2.2 Target users ..... 6
2.3 Gaps in existing tools ..... 6
2.4 Solution objectives ..... 6
**3. Proposed Solution ..... 8**
3.1 Solution overview ..... 8
3.2 Two products, one backend ..... 8
3.3 Feature catalogue ..... 8
3.4 Unique selling points ..... 9
3.5 Expected impact ..... 10
**4. Architecture Overview ..... 11**
4.1 Architectural principles ..... 11
4.2 High-level architecture ..... 11
4.3 End-to-end request lifecycle ..... 13
4.4 Response generation pipeline ..... 14
**5. Detailed Component Architecture ..... 15**
5.1 Backend module map ..... 15
5.2 Restricted execution sandbox (F-01, F-02) ..... 15
5.3 Diagnosis engine ..... 16
5.4 Hint tiers and Hint Guard (F-04, F-05) ..... 17
5.5 One-to-one mentor chat (F-16) ..... 18
5.6 Personalisation architecture (F-17) ..... 19
5.7 Multilingual architecture (F-18) ..... 20
5.8 Voice architecture (F-11, F-12) ..... 21
5.9 Chrome extension architecture (F-13, F-14) ..... 21
5.10 Adaptive learning and memory (F-10, F-15) ..... 22
**6. Data Architecture ..... 23**
6.1 Storage overview ..... 23
6.2 Domain map ..... 23
6.3 Entity relationships ..... 23
6.4 Table catalogue ..... 24
6.5 Data provenance ..... 26
6.6 Retention and privacy ..... 26
**7. API Architecture ..... 27**
7.1 Conventions ..... 27
7.2 Endpoint catalogue ..... 27
7.3 Key contracts ..... 28
**8. AI / ML Architecture ..... 30**
8.1 Where AI is used — and where it is not ..... 30
8.2 Component specification and fallbacks ..... 30
8.3 LLM abstraction ..... 31
8.4 Evaluation framework ..... 31
**9. Technology Stack ..... 32**
9.1 Backend ..... 32
9.2 Frontend, extension and DevOps ..... 32

9.3 Key technologies explained ..... 32
**10. Security Architecture ..... 34**
10.1 Trust boundaries ..... 34
10.2 Threat model ..... 34
10.3 Data protection ..... 34
10.4 Extension security ..... 35
**11. Deployment Architecture ..... 36**
11.1 Two deployment tiers ..... 36
11.2 Packaging ..... 36
11.3 Hosted-backup hardening ..... 37
11.4 Cost ..... 37
11.5 Walking skeleton and demo-day checklist ..... 37
**12. Scalability and Evolution ..... 38**
12.1 Suggested roadmap ..... 38
**13. Implementation Plan ..... 39**
13.1 Team split (4 people) ..... 39
13.2 Timeline ..... 39
13.3 Demo flow (~5 minutes) ..... 39
13.4 Cut order if behind ..... 40
**14. Risks, Assumptions and Open Items ..... 41**
14.1 Risk register ..... 41
14.2 Assumptions ..... 41
14.3 Open items to confirm ..... 41
**Appendix A. Repository Structure ..... 42**
**Appendix B. Glossary ..... 43**

# 1. Executive Summary

**CodeQuest AI** is an AI coding mentor and debugging-mission platform for beginner programmers. It helps a learner understand why their code fails without handing over the answer, and it keeps the learner safe, private and supported in their own language (English, Hindi or Marathi).

The product has two parts that share one backend:

- **CodeQuest Core** — a practice website we own, with challenges, a restricted-execution sandbox, visible and hidden tests, tiered hints and adaptive learning.
- **CodeQuest Companion** — a Chrome (Manifest V3) extension that brings the mentor to any tab, such as LeetCode or GeeksforGeeks. A learner selects code and gets Hint, Debug, Explain, Concept, Listen and Chat, in text and voice.

The central architectural decision is a strict split of responsibilities: **deterministic components decide; AI components only word, translate and converse.** Test outcomes, bug localisation, mistake classification, hint eligibility and mastery updates never depend on an LLM. If every AI service fails during a demo, the platform still diagnoses, hints and explains in all three languages from templates.

| | |
|---|---|
| **Event** | VORTEXA 3.0 Hackathon |
| **Problem statement** | E-02 — AI Coding Mentor & Debugging Missions |
| **Domain** | AI-powered education and learning technology |
| **Primary users** | First- and second-year students learning Python; regional-language learners; self-learners on third-party practice sites |
| **Languages (v1)** | English, Hindi, Marathi — UI, explanations, hints, chat and voice |
| **Build window / team** | ~12 hours · 3–4 people |
| **Distribution** | Local `docker compose up` (primary), hosted backup on EC2; extension distributed as a load-unpacked zip |

## 1.1 What makes the solution different

- **Verifiable diagnosis.** A divergence tracer shows the first step where the learner's variable state differs from a reference run; mutation fingerprinting names the likely mistake class — with zero LLM calls.
- **Enforced hint safety.** A Hint Guard executes and AST-compares any code inside a hint, so "no hint may reveal the full solution" is measured, not hoped for. It applies to hints, chat replies and translated text.
- **Safe by construction execution.** Learner code runs only in an AST-checked, resource-limited, network-less sandbox, and an attack set demonstrates every block.
- **One-to-one private mentor chat** with a restricted tool set and a rule-based fallback.
- **Personalisation that cannot affect grading.** Profile (familiar patterns, coding habits, language, style) changes wording only — never test outcomes, hint tiers, guard thresholds or mastery maths.
- **Trilingual mentor with guard-after-translate**, per-language voices and pre-translated templates so non-English works even without an LLM.

- **Honest labelling** on third-party pages: results are marked `oracle-verified` or `unverified`.

## 1.2 Headline success measures

| Measure | Dataset | Target / reporting |
|---|---|---|
| **Diagnosis accuracy** | ~40 labelled buggy submissions | Per-class accuracy plus “no known pattern” rate |
| **Hint-leak rate** | ~50 red-team prompts in EN / HI / MR, including via chat | Target 0 leaks; report the honest figure and list failures |
| **Attack-block rate** | 6+ sandbox attack snippets | 100% |
| **Hardcode detection** | ~5 hardcoded submissions | Detection rate |
| **Language detection** | ~30 samples incl. code-mixed | Accuracy |
| **Multilingual quality** | ~20 reviewed HI / MR outputs | Reviewer score 1–5 |
| **Personalisation sanity** | Same failure, two profiles | Wording differs; outcomes and tiers identical (automated assertion) |

# 2. Problem Statement and Opportunity

## 2.1 The problem

Beginners learn programming by writing code and reading what goes wrong, but the feedback they receive is hard to use:

- A traceback names an error without explaining it.
- A program can run and still produce a wrong answer, and pass/fail tools do not show where to look.
- Tools that give full answers too early remove the productive struggle; learners copy complete solutions.
- Running code from an unknown author is dangerous — it can read files, open the network or exhaust resources.
- Most tools treat every learner the same, ignore the learner's own coding habits, and explain only in English although many first-year students in India think more easily in Hindi or Marathi.

## 2.2 Target users

| User | Description | Primary need |
|---|---|---|
| **Beginner learner (primary)** | First- or second-year student learning Python | Understand why code fails, without being handed the answer |
| **Regional-language learner** | Student more comfortable in Hindi or Marathi | Explanations, chat and audio in their own language |
| **Self-learner on LeetCode / GfG** | Practises on third-party platforms | A mentor that works on the page they already use |
| **Instructor / judge (secondary)** | Reviews progress and verifies system integrity | Evidence: diagnosis accuracy, hint-leak rate, sandbox blocks |

## 2.3 Gaps in existing tools

| Existing tool behaviour | Gap |
|---|---|
| Online judges report only pass / fail / TLE | No localisation of the bug and no explanation |
| Generic LLM chatbots answer with a full solution | Breaks productive struggle; no guarantee a hint is not a solution |
| Learner-code execution on shared servers | Often unrestricted, or opaque to the learner |
| Progress bars and streaks | Do not track concepts or recurring mistakes |
| Personalisation limited to difficulty | Ignores habits, e.g. always skipping the empty-input case or pasting large blocks |
| English-only explanations | Excludes learners who think in Hindi or Marathi |

## 2.4 Solution objectives

1. Run learner code only inside a restricted sandbox and report per-test outcomes without exposing hidden test contents.
2. Pinpoint where and why code diverges from correct behaviour, deterministically and explainably.

3. Deliver progressive hints that provably never reveal the full solution.
4. Provide a private one-to-one mentor chat, in text and voice, that obeys the same guard rules.
5. Tailor wording to the individual learner’s patterns, habits and language without influencing assessment.
6. Support English, Hindi and Marathi end to end, including voice.
7. Work on the sites learners already use through a Chrome extension, and be transparent about what can be verified.
8. Keep the core demo independent of external services, and stay within a zero-cost infrastructure budget.

# 3. Proposed Solution

## 3.1 Solution overview

A restricted-execution practice environment is combined with a mentor layer in which diagnosis is deterministic and the LLM is used only for wording. The end-to-end behaviour is:

1. Learner code runs only in the sandbox. Each test reports `passed`, `wrong_output`, `runtime_error` or `timeout`; hidden test content is never shown.
2. On failure, a **divergence tracer** runs the reference and the learner's code side by side and finds the first step where variable state differs.
3. **Mutation fingerprinting** matches the failure pattern to a known mistake class (off-by-one, missing empty-input check, wrong initial value).
4. A **Hint Guard** enforces four hint tiers and blocks any hint that would reveal a full solution.
5. A **one-to-one mentor chat** lets the learner talk to the mentor privately, by text or voice, within the same guard rules.
6. A **learner profile** shapes how things are explained, never whether a test passes or which hint level is allowed.
7. Everything is available in **English, Hindi and Marathi**, with ElevenLabs voice.
8. A Chrome extension carries the mentor to any tab; a dashboard shows a skill radar, mistake heatmap and recurring-mistake recall.

## 3.2 Two products, one backend

| Aspect | CodeQuest Core (web) | CodeQuest Companion (extension) |
|---|---|---|
| **Where it runs** | Our own React site | Any browser tab (LeetCode, GfG, our site) |
| **Problem source** | Our own challenges with reference solutions, visible and hidden tests | Page-visible problem text and examples; reference built by an LLM "oracle" and validated |
| **Verification** | Fully verified — own tests and reference | Labelled `oracle-verified` or `unverified` |
| **Interaction** | Editor, results, hint ladder, trace panel, chat page, dashboard | Floating toolbar, result card, side-panel chat, popup dashboard |
| **Shared backend** | Same FastAPI services for diagnosis, guard, chat, personalisation, i18n and voice | Same |

## 3.3 Feature catalogue

| ID | Feature | Purpose | Key technologies |
|---|---|---|---|
| **F-01** | Restricted execution sandbox | Run untrusted code without file, network or unbounded CPU/memory | subprocess, rlimits, AST allowlist, Docker |
| **F-02** | Per-test outcomes with hidden tests | Show which tests failed without leaking hidden content | FastAPI, sandbox harness, SQLite |
| **F-03** | Plain-language failure explanation | Turn tracebacks and wrong output into understandable text | Template engine, LLM wrapper, i18n |
| **F-04** | Tiered progressive hints | Each hint more specific than the last; never the solution | Hint Guard, LLM, SQLite |

| ID | Feature | Purpose | Key technologies |
|---|---|---|---|
| **F-05** | Hint Guard | Enforce hint safety by execution and AST checks | AST similarity, sandbox re-run, regex |
| **F-06** | Divergence tracer | Locate the first step where state differs | `sys.settrace`, locals snapshots |
| **F-07** | Mutation fingerprinting | Name the mistake class without an LLM | AST mutation, Hamming distance |
| **F-08** | Fuzzed hidden tests + hardcode detector | Prevent memorising tests and `if input == X` cheats | Differential fuzzing, shrinker |
| **F-09** | Effort-aware unlocking + copy detection | Stop click-through hints and pasted solutions | Editor events, SQLite |
| **F-10** | Adaptive learning model | Target weakest concepts; recall repeated mistakes | Elo / BKT, SQLite |
| **F-11** | Voice output (Listen) | Hear approved explanations in own language | ElevenLabs TTS, disk cache |
| **F-12** | Explain-back (rubber duck) | Prove understanding of the fix before mastery moves | ElevenLabs Scribe, LLM rubric |
| **F-13** | Chrome extension companion | Mentor on any tab | MV3, Shadow DOM, service worker |
| **F-14** | Oracle verification for foreign problems | Validate verdicts on pages we do not own | LLM reference, sandbox, fuzzing |
| **F-15** | Dashboard and per-user memory | Progress, radar, heatmap, “seen before” | React, Recharts, Chroma |
| **F-16** | One-to-one mentor chat | Private follow-up conversation with guarded tools | FastAPI SSE, SQLite, Chroma, Guard |
| **F-17** | User-specific answers | Use familiar patterns, habits and style | AST features, profile tables |
| **F-18** | Multilingual support | EN / HI / MR text and voice, guard after translate | i18next, glossary, ElevenLabs |

## 3.4 Unique selling points

| # | USP | Why it matters |
|---|---|---|
| **1** | Divergence Tracer | Deterministic, verifiable bug localisation; the LLM only words the result |
| **2** | Mutation Fingerprinting | Explainable recurring-mistake detection with zero LLM calls; yields trustworthy accuracy numbers |
| **3** | Hint Guard | Red-team-able guarantee against solution leakage, including AI-generated hints |
| **4** | Visible sandbox attack set | Safety demonstrated, not claimed (`/etc/passwd`, socket, fork bomb, `while True`, 5 GB allocation) |
| **5** | Fuzzed hidden tests + hardcode detector | Hidden tests cannot be memorised or leaked; cheating is flagged |
| **6** | Explain-back gating | Mastery updates only if the learner explains the fix aloud; voice has a real job |
| **7** | Private mentor chat with guarded tools | Chat is safe by construction because every reply passes the guard |
| **8** | Habit- and pattern-aware personalisation | Explanations use constructs the learner already knows; cannot alter grading |

| # | USP | Why it matters |
|---|---|---|
| **9** | Trilingual mentor (guard-after-translate) | Real inclusion for Hindi and Marathi learners, text and voice |
| **10** | Extension on any tab with verified / unverified labels | Works where learners practise and is honest about what it can verify |
| **11** | Per-user vector memory | “Same mistake as in Two Sum last Tuesday” — influences wording only |
| **12** | No-LLM fallback | Product keeps working if an API fails mid-demo; supports offline classrooms |
| **13** | Credit-aware ElevenLabs usage | Cache, credit meter and pre-rendered demo audio stretch the free credits |

## 3.5 Expected impact

| Outcome | How it is measured |
|---|---|
| **Learners find bugs faster** | Time-to-fix and attempts-to-pass on repeat mistake classes |
| **Fewer copied solutions** | Share of passes preceded by a large paste that are then verified by explain-back |
| **Trustworthy mentor** | Hint-leak rate on a red-team set (target 0; report the honest figure) |
| **Accurate diagnosis** | Diagnosis accuracy on ~40 labelled buggy submissions |
| **Safe execution** | Attack-block rate on 5+ attack snippets (target 100%) |
| **Language access** | Explanation quality in Hindi / Marathi on a reviewed sample; share of sessions using non-English |

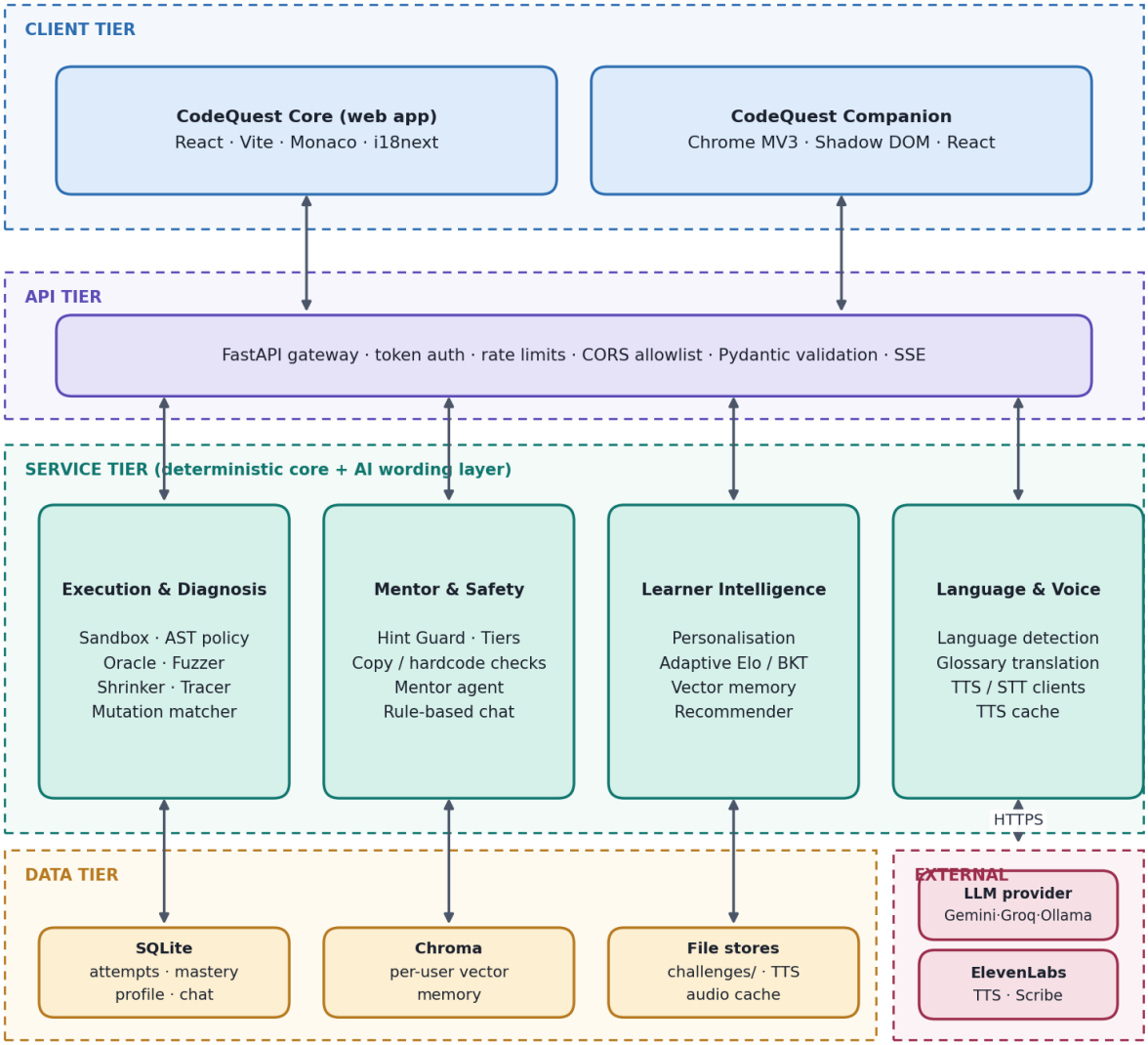
# 4. Architecture Overview

## 4.1 Architectural principles

| Principle | Meaning | How it is enforced |
|---|---|---|
| **Deterministic core, AI wording layer** | Outcomes, diagnosis, tiers and mastery never depend on an LLM | Sandbox, tracer, fingerprint matcher and Elo/BKT are algorithmic; LLM output is only rendered text |
| **Guard on every output path** | No solution leaks via hints, chat, translation or voice | Hint Guard before release; re-run after translation; `/tts` accepts only approved text |
| **Language-neutral facts first** | Diagnosis and guard operate on structured data, then wording is rendered in the target language | Neutral diagnosis object → render → guard → translate → guard |
| **Personalisation is wording-only** | Profile may change analogies, order and nudges | Hard rule plus automated assertion test (same failure, two profiles, identical outcomes and tiers) |
| **Graceful degradation** | Core demo must not depend on external services | Template hints, rule-based chat, pre-translated locales, browser TTS, local Ollama, cached audio |
| **Least privilege and no secrets in clients** | Extension and web hold no API keys | All external calls via the backend; content script talks only to the service worker |
| **Local-first privacy** | No accounts or emails; learner owns data | Generated user id, view / edit / delete profile, `DELETE /users/{id}/data` |
| **Zero-cost by design** | Free tiers and open components only | SQLite, Chroma, MiniLM, Gemini / Groq free tier, Ollama, load-unpacked extension |

## 4.2 High-level architecture

The system is organised in five tiers. Clients call a single FastAPI gateway; behind it four service groups cooperate over a shared data tier; external AI and voice providers are reached only from the backend.




*Clients hold no API keys and no business logic. All LLM / voice calls are made by the backend only.*

*Deterministic components decide (tests, diagnosis, tiers, mastery); AI components only word, translate and converse.*

*Figure 1. High-level system architecture*

| Tier | Responsibility | Main components |
|---|---|---|
| **Client** | Capture code and intent; present results; play audio | React web app (Monaco, Recharts, i18next); Chrome MV3 extension (content script, service worker, offscreen document, side panel) |
| **API** | Authentication, rate limiting, CORS, validation, streaming | FastAPI + Uvicorn, Pydantic schemas, SSE |
| **Service** | Deterministic execution and diagnosis; guarded mentoring; learner modelling; language and voice | sandbox, oracle, tracer, mutants, guard, agent, personalization, adaptive, memory, i18n, voice, llm modules |
| **Data** | Durable state and caches | SQLite (structured), Chroma (vectors), disk folders (challenges, TTS cache) |
| **External** | Wording, translation, speech | Gemini or Groq (Ollama fallback); ElevenLabs TTS and Scribe |

## 4.3 End-to-end Detailed Flow lifecycle

The most important flow is a code submission that fails. The figure below traces it from the editor to the learner's screen; the numbered steps are described in the table that follows.

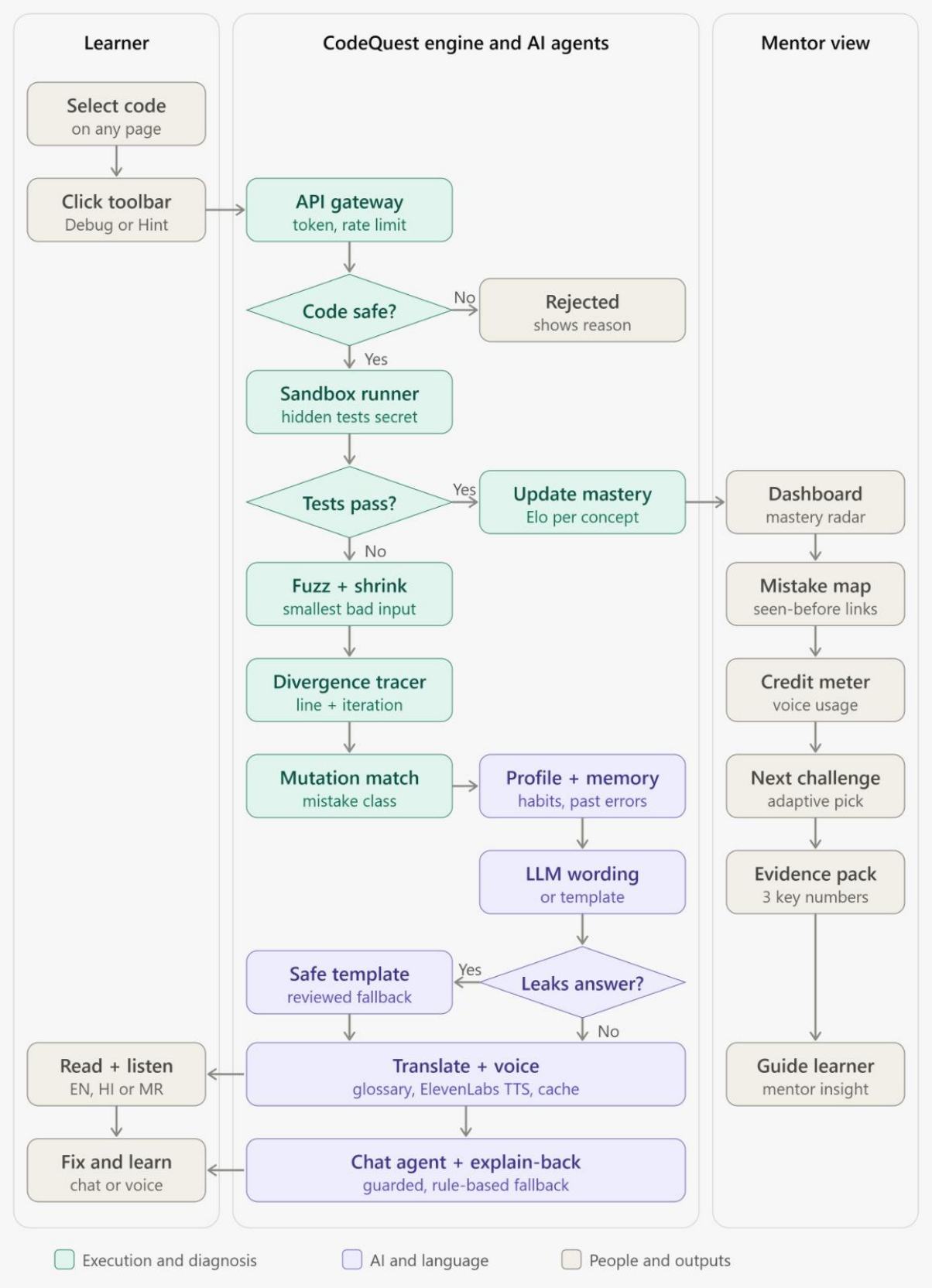
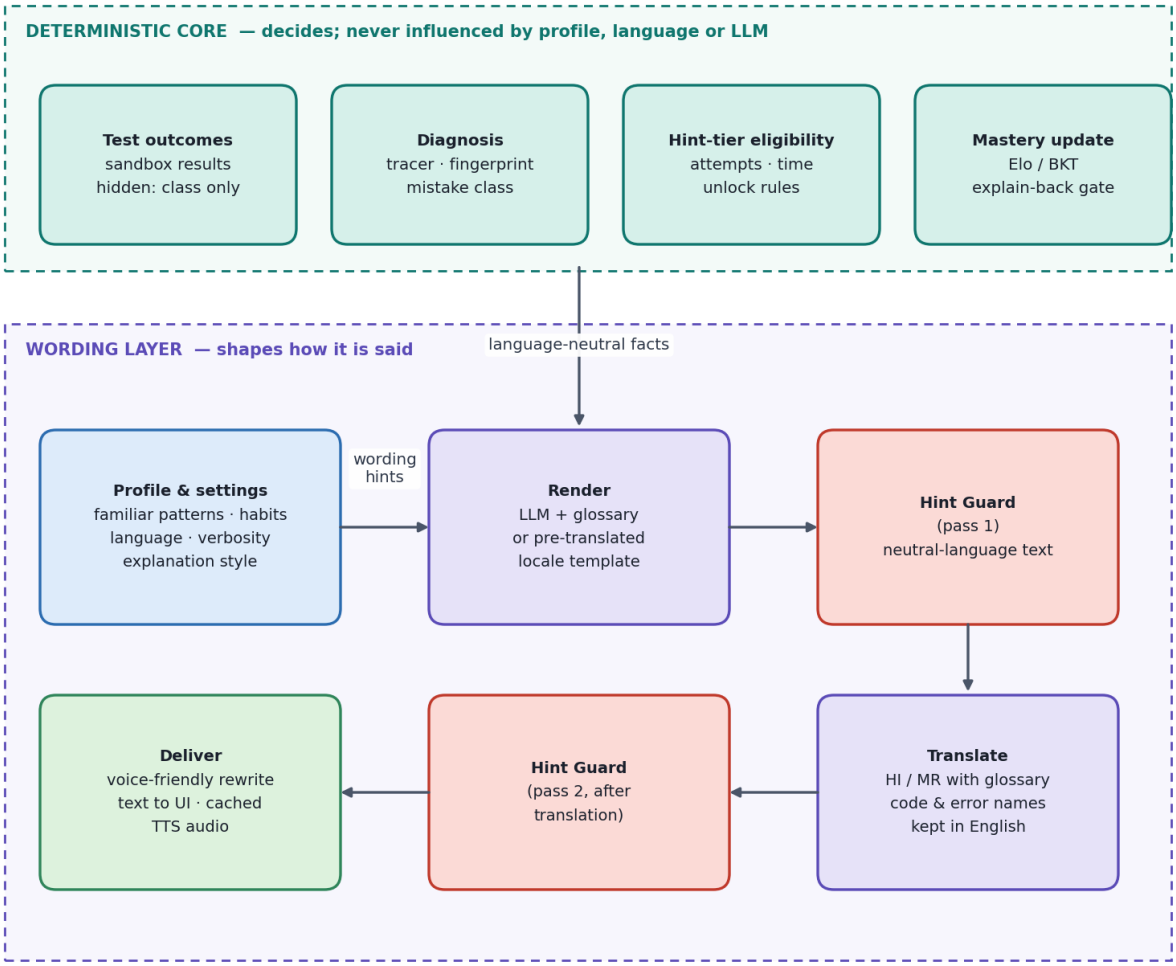



*Figure 2. Flow lifecycle pipeline*

## 4.4 Response generation pipeline

Every explanation, hint and chat reply passes through the same two-layer pipeline. The deterministic core produces language-neutral facts; the wording layer then personalises, renders, guards, translates, guards again and finally prepares the text for voice. Nothing in the wording layer can write back into the core.




5. Detailed Component Architecture

## 5.1 Backend module map

The backend is a single FastAPI application organised as cohesive packages. Routers are thin; logic lives in the service packages below.

| Package | Responsibility | Key modules |
|---|---|---|
| **routers/** | One file per API group; request validation and response shaping | health, users, challenges, submissions, analyze, hints, explain, chat, voice, explainback, dashboard |
| **sandbox/** | Restricted execution of learner code | runner, harness, ast_policy, attacks/ |
| **oracle/** | Reference building and differential testing | oracle_gen, generators, fuzzer, shrinker |
| **tracer/** | Step-level execution comparison | trace_runner, divergence |
| **mutants/** | Mistake classification by fingerprint | operators, fingerprint, matcher |
| **guard/** | Hint safety, tiers and integrity checks | hint_guard, tiers, hardcode_detector, copy_detector |

| Package | Responsibility | Key modules |
|---|---|---|
| **agent/** | One-to-one mentor chat | mentor_agent, tools, chat_service, rule_based_chat |
| **personalization/** | Learner profile from code and behaviour | ast_features, habit_tracker, profile_service, style_adapter |
| **i18n/** | Language detection, glossary, translation, voice text | detect, glossary.json, translate, voice_text, locales/{en,hi,mr}.json |
| **voice/** | ElevenLabs TTS / Scribe, cache, credit accounting | elevenlabs_client, cache, voices |
| **llm/** | Single provider-agnostic entry point and prompts | client (Gemini / Groq / Ollama), prompts/ |
| **memory/** | Local embeddings and per-user vector store | embeddings (all-MiniLM-L6-v2), vector_store (Chroma) |
| **adaptive/** | Mastery model and next-challenge recommendation | mastery (Elo / BKT), recommender |
| **db/, schemas/** | Persistence models, DDL, seed data, Pydantic contracts | models, schema.sql, seed |

## 5.2 Restricted execution sandbox (F-01, F-02)

The sandbox is the first line of defence and is never cut from the build. It combines a static pre-check with layered runtime isolation.

| Layer | Control | Purpose |
|---|---|---|
| **1. Static policy** | AST allowlist: only `math, collections, heapq, itertools, functools, bisect, typing`; reject `open, exec, eval, __import__` and dunder attribute access | Block dangerous code before it runs |
| **2. Interpreter hardening** | `python -I` (isolated mode), empty environment, temporary working directory | No ambient configuration, no access to project files |
| **3. Resource limits** | `resource.setrlimit` for CPU, memory, file size and process count; wall-clock timeout; kill the whole process group | Stop infinite loops, fork bombs and large allocations |
| **4. Container isolation** | Docker `--network none --memory 256m --cpus 1 --pids-limit 64`; non-root user; read-only filesystem with tmpfs | No network, bounded resources, immutable image |
| **5. Output discipline** | Harness wraps the learner function, calls it with test inputs and emits JSON; hidden inputs and expected values are never serialised to the client | Prevent test leakage |

**Platform note.** The `resource` module is unavailable on Windows, so on Windows hosts the runner is Docker-only. This is decided at hour 0 of the build.

**Outcome model.** Each test yields exactly one of `passed`, `wrong_output`, `runtime_error` or `timeout`. For visible tests the learner sees input, expected and actual values; for hidden and fuzzed tests only the outcome class and a failure category (for example “empty input” or “boundary case”) are returned.

**Attack demonstration.** `backend/app/sandbox/attacks/` holds six or more snippets — file read of `/etc/passwd`, socket open, fork bomb, `while True`, 5 GB allocation and import abuse. `POST /admin/attacks` runs them and returns a block reason for each, giving a measurable attack-block rate.

## 5.3 Diagnosis engine

### 5.3.1 Differential fuzzing, shrinking and hardcode detection (F-08)

- **Generators** are signature-driven and edge-biased: empty input, single element, duplicates, negatives and large values. Each attempt uses a freshly seeded set, so hidden tests cannot be memorised.
- **Differential execution** compares the learner function against the reference on every generated input.
- **Shrinking** repeatedly simplifies a failing input (dropping elements, reducing values) while it still fails, producing the smallest understandable counterexample.
- **Hardcode detector** uses AST pattern checks to flag branches such as `if input == X: return Y`, setting `hardcode_flag` on the attempt.

### 5.3.2 Divergence tracer (F-06)

The tracer registers a `sys.settrace` hook that records `(line number, snapshot of local variables)` on each step for both the reference and the learner code, run on the same minimal failing input.

1. Run reference and learner under tracing with a step cap (tracing is slow).
2. Align the two traces by shared variable names.
3. Report the first step at which a shared variable differs: iteration, variable, expected value, actual value and line, e.g. “On iteration 3, `total` is 7 expected and 5 in yours (line 6)”.
4. **Fallback:** if variable names do not overlap, compare final output and loop iteration counts instead.

### 5.3.3 Mutation fingerprinting (F-07)

Mutation fingerprinting works in two phases so that online classification is cheap and needs no LLM.

| Phase | Activity |
|---|---|
| **Offline (per challenge)** | AST mutation operators (`<` → `<=`, `range(n)` → `range(n-1)`, initial-value changes and similar) are applied to the reference to create mutants that mimic beginner bugs. Each mutant runs on a fixed fuzz set, producing a pass/fail bit string — its fingerprint — labelled with a mistake class. Stored in `mutants`; pre-built by `scripts/generate_mutants.py`. |
| **Online (per submission)** | The learner’s code runs on the same fixed fuzz set to produce its own vector. The nearest stored fingerprint by Hamming distance names the likely mistake and gives a confidence. Below a threshold the result is “no known pattern”. |

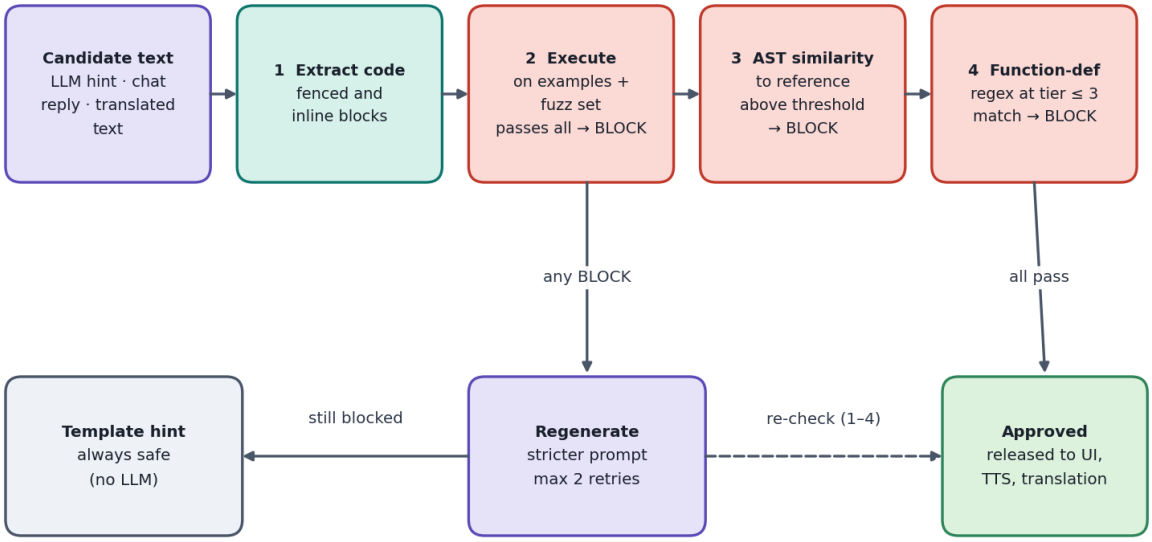
### 5.3.4 Copy detection and effort-aware unlocking (F-09)

Editor events (paste size, timing, first run, edit bursts) are stored as `habit_events`. A large paste followed by a pass sets `large_paste_flag` and triggers an explain-back requirement before mastery is credited. Hints unlock by attempts and elapsed time (section 5.4), discouraging click-through.

## 5.4 Hint tiers and Hint Guard (F-04, F-05)

Hints are released in four tiers, each more specific than the last. Every candidate hint — whether produced by an LLM, a template or a translation — must pass the Hint Guard before it reaches the learner.

| Tier | Content | Unlock rule (starting values; tune later) |
|---|---|---|
| **1 Concept** | Which idea is relevant | Immediately |
| **2 Location** | Where to look, from the tracer: “iteration 3, variable `total`” | ≥ 2 attempts or ≥ 3 min |
| **3 Pattern** | The same idea shown on a different example | ≥ 3 attempts or ≥ 6 min after tier 2 |
| **4 Gapped pseudocode** | Steps with blanks | ≥ 4 attempts or ≥ 10 min after tier 3 |




*The same four checks run again after translation (HI / MR) and on every chat reply, so no output path bypasses the guard.*

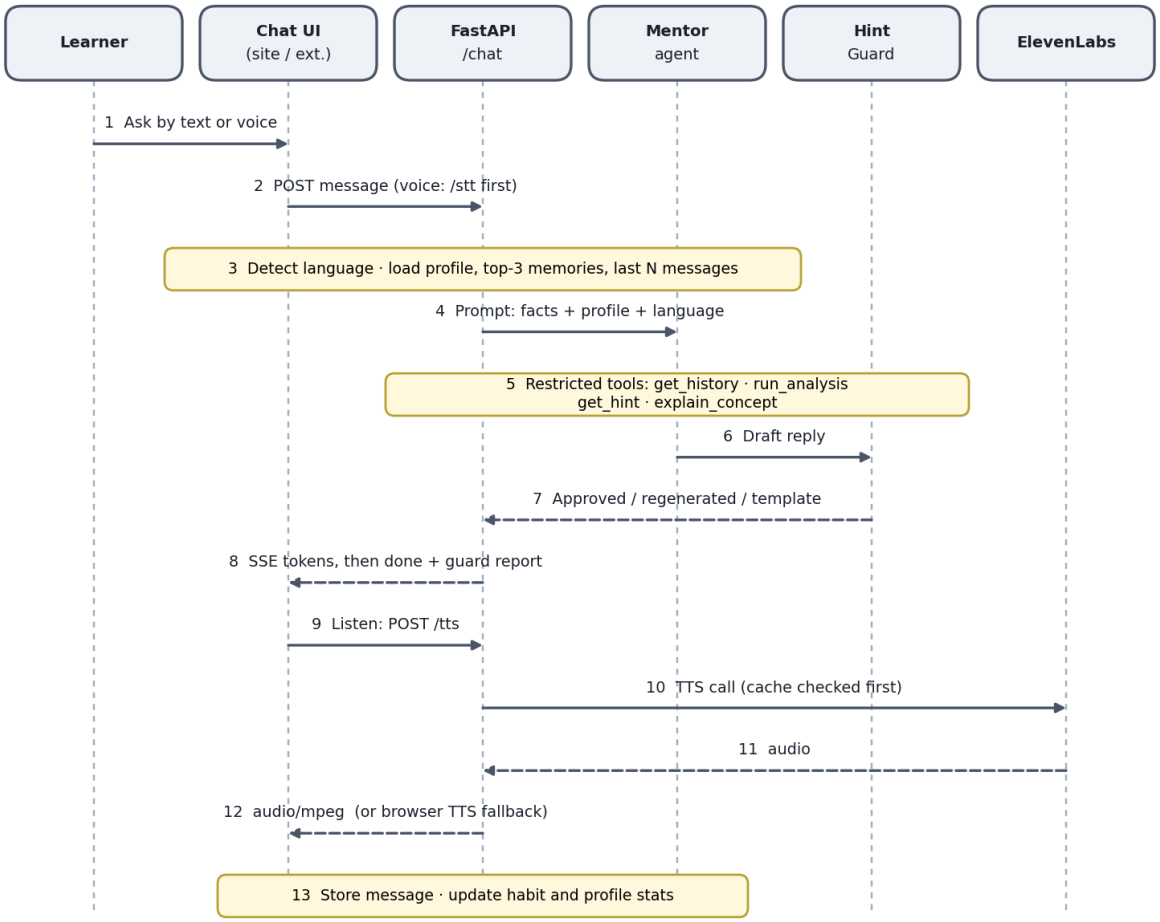
*Figure 4. Hint Guard checking pipeline*

| Step | Check | Block condition |
|---|---|---|
| **1** | Extract fenced and inline code from the candidate text | — |
| **2** | Execute extracted code on examples and the fuzz set in the sandbox | Passes all tests |
| **3** | AST similarity to the reference, with normalised variable names | Above threshold (start 0.7; tune on the red-team set) |
| **4** | Full-function-definition regex at tier ≤ 3 | Match |
| **5** | Regenerate with a stricter prompt | Maximum 2 retries |
| **6** | Template fallback | Always safe |
| **7** | Repeat steps 1–4 after translation (multilingual) | Same rules |

Every guard decision is recorded in `hints_issued` and `chat_messages` (`guard_passed`, `guard_retries`, `source`). The `/admin/redteam` endpoint replays the red-team prompt set and returns the leak rate per language, which is the headline evidence for judges.

## 5.5 One-to-one mentor chat (F-16)

Chat is a private conversation between one learner and one AI mentor. Threads belong to a single learner and may be attached to a problem; there is no group chat or sharing between users.




*On LLM failure step 4–6 are replaced by the rule-based intent engine; the guard and storage steps still run.*

*Figure 5. Chat message sequence with guard and voice*

## Chat rules

1. Every assistant message passes through the Hint Guard. A request such as “just give me the answer” is declined kindly in the learner’s language and answered with the next allowed hint tier.
2. The agent may call only four tools; none returns reference code.
3. Context is assembled in a fixed order: system rules → language and style from settings → profile summary → top-3 retrieved diagnoses → last N messages → current message and optional selected code.
4. Replies are language-aware: if the learner writes Marathi, the mentor answers in Marathi (following the thread / mentor language setting).
5. Content from web pages, problem text and tool outputs is treated as data and cannot change system rules or unlock hint tiers (prompt-injection defence).
6. The learner can delete a thread or all history at any time.
7. If the LLM is unavailable, chat switches to the rule-based engine and the `done` event carries `"mode":"rule_based"`.

| Tool | Purpose | Safety property |
|---|---|---|
| `get_history(user, concept)` | Recall the learner’s earlier attempts and diagnoses | Scoped by `user_id` |
| `run_analysis(code)` | Run sandbox, tracer and fingerprint on selected code | Same AST policy and limits as submissions |

| Tool | Purpose | Safety property |
|---|---|---|
| `get_hint(level)` | Fetch a guarded hint at an allowed tier | Respects unlock state; guard-wrapped |
| `explain_concept(name)` | Explain a concept in the learner's language and style | Output guard-checked |

**Streaming.** Replies are delivered over Server-Sent Events (SSE). Text is buffered through the guard so that streaming starts only after the first guarded segment is approved, or proceeds guarded sentence by sentence. Assumption: "one-to-one" means learner ↔ AI mentor; a human-mentor channel is supported later by the `participant_type` column on `chat_threads`.

**Rule-based fallback.** `rule_based_chat.py` matches intents (hint, explain, concept, progress) and answers from templates and diagnosis data, so chat degrades rather than fails.

## 5.6 Personalisation architecture (F-17)

A learner profile is built from deterministic features, so no LLM is needed to create it. It is viewable, editable and deletable by the learner.

| Profile dimension | How it is learned | How it is used |
|---|---|---|
| **Familiar patterns** | AST counts of constructs used successfully (for-loops vs comprehensions, dict / set, recursion, two-pointer, sorting) | Choose analogies and examples in constructs the learner already knows; avoid unfamiliar ones at early tiers |
| **Coding habits** | Naming style, comment density, function length, edge-case skip rate, paste frequency, time-to-first-run, run-before-think ratio | Targeted nudges ("you often skip the empty list"), hint pacing, copy-detection sensitivity |
| **Recurring mistakes** | Mistake-class counters plus vector recall | "Same boundary issue as in Two Sum" |
| **Programming language** | Explicit setting (Python for challenges; Java / C++ / JS recognised in the extension for explanation only) | Syntax of examples |
| **Natural language** | Explicit setting plus detection | Language of all outputs |
| **Explanation style** | Explicit (short / detailed, analogy / formal) and implicit (which explanations preceded a pass) | Verbosity and tone |

| Signal | Extraction | Effect on output |
|---|---|---|
| **Edge-case skip rate** | Fraction of failures in empty / single-element classes | Early nudge: "check the empty list first" |
| **Paste rate** | Paste events per attempt | Raises explain-back probability |
| **Run-before-think ratio** | Time to first run vs problem length | Suggests a short "predict the output" step |
| **Naming / comment style** | Simple metrics | Advice on descriptive names only if the learner uses single-letter names |
| **Analogies that preceded a pass** | Link hints / explanations to next-attempt outcome | Prefer those analogy tags |

> **Hard constraint**
>
> Outputs of the personalisation module are inputs to wording only. They cannot change test outcomes, hint-tier eligibility, guard thresholds or mastery maths. This keeps the system auditable and prevents any perception of grading bias.

## 5.7 Multilingual architecture (F-18)

English, Hindi and Marathi are supported in the first release; adding a language is a data task (a locale pack, glossary entries and a voice id).

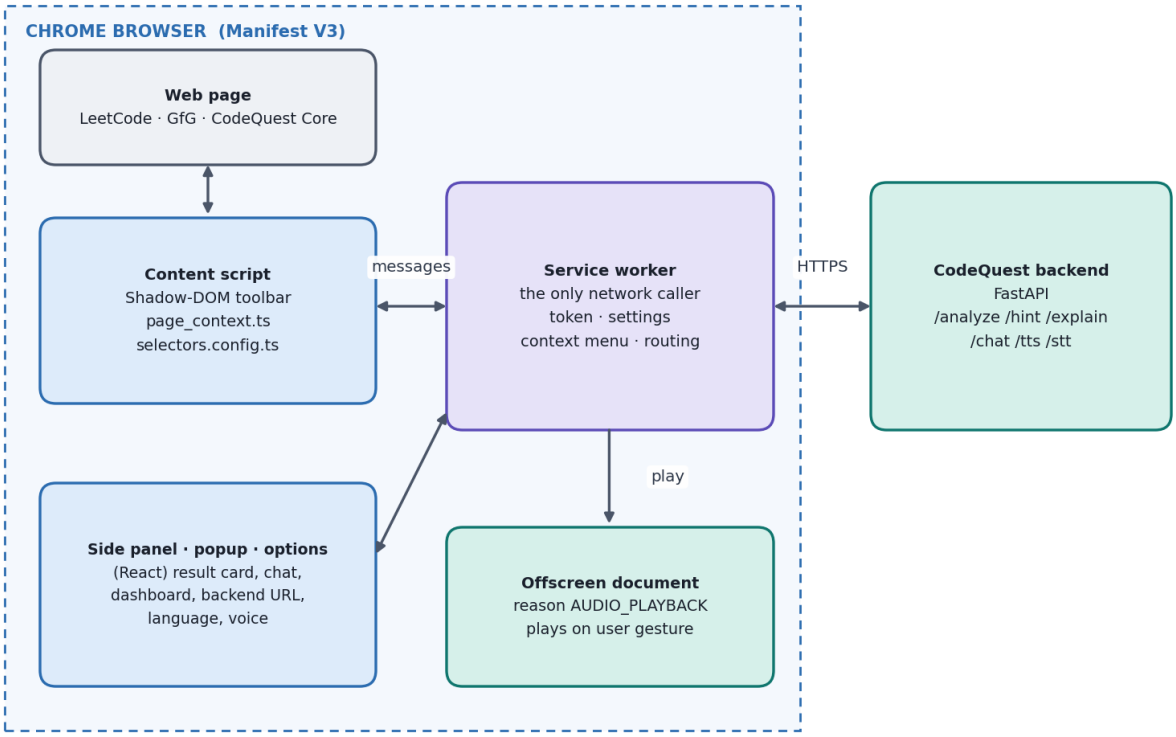
| Aspect | Design |
|---|---|
| **Order of operations** | Diagnosis and guard run on language-neutral structured data (mistake class, tier, tracer step); wording in the target language is generated afterwards. |
| **Guard after translation** | Extracted code blocks are checked again after translation so translation cannot smuggle in a solution. |
| **Glossary** | Terms such as loop, index, array, variable stay in English or appear as "लूप (loop)" so learners can map to documentation. Code, identifiers and error names (e.g. `IndexError`) are never translated. |
| **Detection** | Unicode script check separates Latin from Devanagari. Hindi vs Marathi is decided by the user's setting first, then a stop-word heuristic (e.g. Marathi आहे / होते / मला; Hindi है / था / मुझे). |
| **Code-mixed input** | Romanised input such as "mala loop madhe error yet ahe" is handled by the LLM; the reply follows the dominant language of the user's setting. |
| **No-LLM fallback** | Template explanations are pre-translated in `locales/en.json`, `hi.json`, `mr.json` so non-English works offline. |
| **Voice** | Scribe transcribes in the chosen language; TTS uses a per-language voice; browser `speechSynthesis` is the fallback. |
| **Quality control** | A bilingual teammate reviews ~20 outputs and records findings in `multilingual_eval.md`. Known risk: LLM Marathi quality may be weaker than Hindi, so the glossary and templates carry the highest-value messages. |

## 5.8 Voice architecture (F-11, F-12)

| Use | Endpoint | When | Credit control |
|---|---|---|---|
| **Listen on explanations, hints, chat** | TTS | On click only | Cache by `sha256(text\|voice\|lang\|model)`; daily character cap per token |
| **Demo showcase clips (EN / HI / MR)** | TTS (higher-quality model) | Pre-rendered before the event | Served from cache at zero cost |
| **Explain-back and voice chat** | Scribe | After a fix or when the mic is pressed | Duration limit (60 s); audio discarded after transcription |
| **Optional voice mentor** | ElevenLabs Agents | Last priority; first cut | Speaks only guard-approved text; tools limited to `get_hint(level)` and `run_tests()` |

- Only finished, guard-approved text is spoken; a voice-friendly rewrite converts symbols ("<=" → "less than or equal to").
- `/tts` accepts text only if it matches a short-lived approved-text registry or hash.
- Stock or designed voices only — no cloning of real people. No core flow depends on ElevenLabs.
- ElevenLabs model names, Marathi / Hindi coverage and credit rates are marked `[VERIFY]` and must be confirmed on the provider dashboard before reliance.

## 5.9 Chrome extension architecture (F-13, F-14)




*No API keys, no scoring logic and no direct backend calls in the content script.*

*Bundled code only (no remote scripts); fixed manifest key keeps the extension ID stable for the CORS allowlist.*

*Figure 6. Chrome extension (Manifest V3) component architecture*

| Component | Role | Notes |
|---|---|---|
| **Content script** | Detects selection, shows the floating toolbar (Hint, Debug, Explain, Concept, Listen, Chat), reads the visible problem statement | Rendered in Shadow DOM so host-page CSS cannot break it; per-site selectors in `selectors.config.ts` with two fallbacks |
| **Service worker** | The only network caller; holds settings, token and context menu | Short-lived in MV3; keeps API keys and CORS issues out of web pages |
| **Offscreen document** | Audio playback (reason `AUDIO_PLAYBACK`) | Service workers have no DOM or audio; playback starts from a user gesture to avoid autoplay blocking |
| **Side panel, popup, options** | Result card, chat side panel, dashboard (radar, heatmap, credit meter), backend URL / language / voice / token | React; same i18n locale files as the web app |

### Oracle verification for foreign problems (F-14)

On third-party pages we own neither tests nor a reference. The backend asks an LLM for a brute-force reference, validates it against the page's visible examples, and only then fuzzes the learner's code against it.

- Oracle passes the visible examples → fuzz, trace and fingerprint; result labelled **oracle-verified**.
- Oracle fails the visible examples → explanation-only mode, no fuzzing; result labelled **unverified**, and the UI says so.

- Only the problem text the user chose to analyse is used; just a hash is stored, nothing is scraped in bulk, and the extension is disabled on live contest pages.

## 5.10 Adaptive learning and memory (F-10, F-15)

- **Mastery model.** An Elo-style (or BKT) update per concept, weighted by problem difficulty: success on a hard problem raises mastery more than success on an easy one. Scores start at 1000 and are stored in `concept_mastery`.
- **Gating.** Mastery is credited only when the copy check or explain-back requirement is satisfied.
- **Recommender.** `adaptive/recommender.py` picks the next challenge for the weakest concept and explains why ("Weakest concept: boundaries (score 940)").
- **Vector memory.** Each diagnosis (problem title, mistake class, divergence summary, concepts) is embedded locally with `all-MiniLM-L6-v2` and stored in a Chroma collection per user. The top-3 similar diagnoses power "same mistake as before" recall and analogy selection. Memory influences wording only, and learners can disable it with `memory_enabled`.
- **Dashboard.** Skill radar, mistake heatmap, recent diagnoses with "seen before" links, credit meter and verified / unverified breakdown.

# 6. Data Architecture

## 6.1 Storage overview

| Store | Technology | Holds | Notes |
|---|---|---|---|
| **Structured data** | SQLite on a mounted volume | Users, settings, profiles, attempts, results, diagnoses, mastery, hints, chat, voice usage, caches | Timestamps are ISO-8601 UTC text; booleans are INTEGER 0/1; `PRAGMA foreign_keys=ON` |
| **Vector memory** | Chroma on a volume | Per-user diagnosis embeddings | Collection `diagnoses_<user_id>`; ids referenced from SQLite `diagnoses.vector_id` |
| **File stores** | Disk folders | `challenges/` content and the TTS audio cache | Reference solutions and hidden tests stay server-side |

## 6.2 Domain map

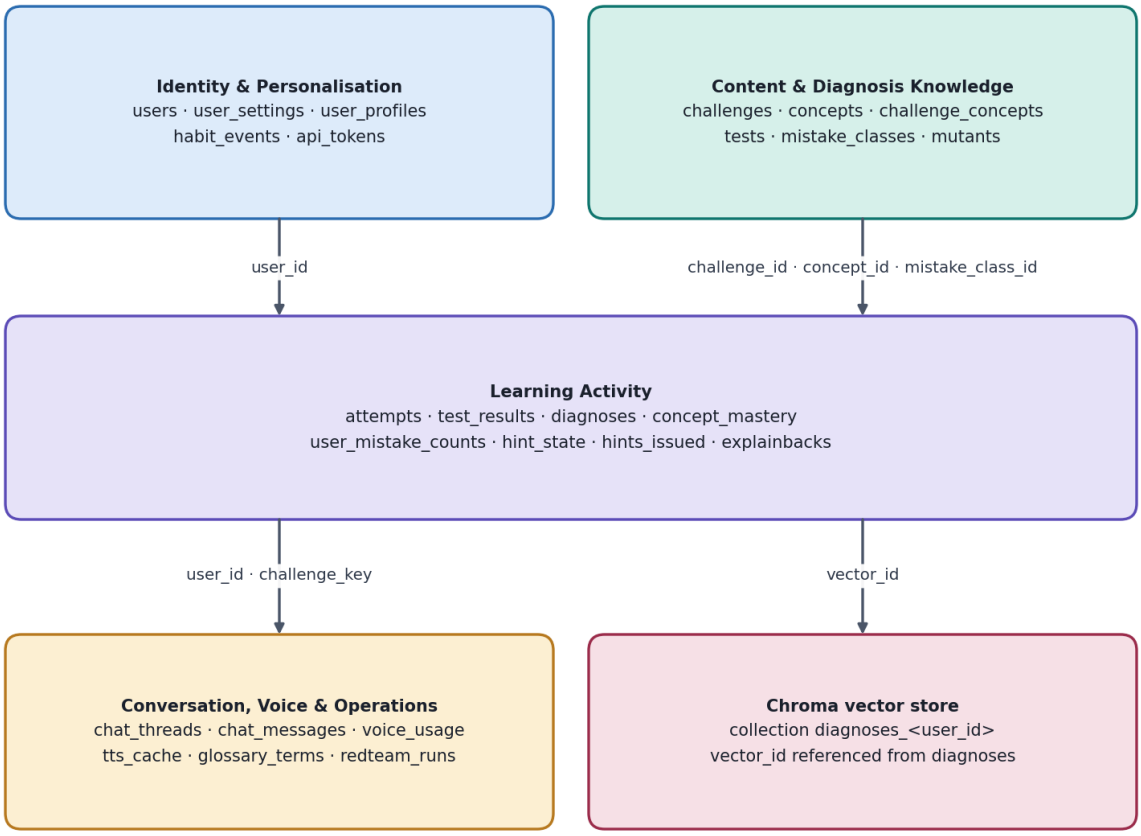



*Figure 7. Data domains and their main links*

## 6.3 Entity relationships

| Parent | Child | Cardinality | Key |
|---|---|---|---|
| **users** | user_settings, user_profiles | 1 : 1 | `user_id` |
| **users** | habit_events, attempts, hint_state, explainbacks, chat_threads, voice_usage | 1 : many | `user_id` |

| Parent | Child | Cardinality | Key |
|---|---|---|---|
| **challenges** | tests, mutants, attempts | 1 : many | `challenge_id` |
| **challenges ↔ concepts** | challenge_concepts | many : many | `(challenge_id, concept_id)` |
| **attempts** | test_results | 1 : many | `attempt_id` |
| **attempts** | diagnoses | 1 : 0..1 | `attempt_id` (UNIQUE) |
| **mistake_classes** | mutants, diagnoses, user_mistake_counts | 1 : many | `mistake_class_id` |
| **users ↔ concepts** | concept_mastery | many : many | `(user_id, concept_id)` |
| **hint_state** | hints_issued | 1 : many | `(user_id, challenge_key)` |
| **chat_threads** | chat_messages | 1 : many | `thread_id` |

Deletion cascades from `users` (`ON DELETE CASCADE`), so `DELETE /users/{id}/data` removes SQL rows, the Chroma collection and cached user audio references in one operation.

## 6.4 Table catalogue

### Identity and personalisation

| Table | Purpose | Key columns and constraints |
|---|---|---|
| **users** | Local learner identity (no accounts) | `id` UUID PK; `created_at`; `last_seen_at`; `deleted_at` |
| **user_settings** | Language, style, voice preferences | PK/FK `user_id`; `ui_language`, `mentor_language` CHECK in (en, hi, mr); `auto_detect_language`; `programming_language`; `verbosity` (short / detailed); `explanation_style` (analogy / formal); `voice_enabled`; `voice_id_override`; `memory_enabled` |
| **user_profiles** | Learned profile | PK/FK `user_id`; `familiar_patterns` JSON; `habit_metrics` JSON; `preferred_analogies` JSON; `summary_text`; `updated_at` |
| **habit_events** | Behavioural events for habits and copy detection | `id` PK; FK `user_id`, `attempt_id` (SET NULL); `event_type` in (paste, first_run, edit_burst, run, hint_request); `value`; index `(user_id, created_at)` |
| **api_tokens** | Hashed install tokens and quotas | PK `token_hash`; `daily_char_cap` default 20,000; `rate_limit_per_min` default 30; `revoked` |

### Content and diagnosis knowledge

| Table | Purpose | Key columns and constraints |
|---|---|---|
| **challenges** | Challenge metadata | `id` PK; `difficulty` CHECK 1–5; `signature`; `description_en/hi/mr`; `starter_code`; `reference_path` (server-side only); `is_foreign` |
| **concepts** | Concept taxonomy | `id` PK (e.g. loops, boundaries, hashmap); `name_en/hi/mr` |
| **challenge_concepts** | Challenge ↔ concept link | PK `(challenge_id, concept_id)`; both FK with cascade |
| **tests** | Visible and hidden tests | `kind` in (visible, hidden); `input_json`; `expected_json`; `failure_category`; index `(challenge_id, kind)` |
| **mistake_classes** | Mistake taxonomy and templates | `id` PK (off_by_one_bound, missing_empty_case, wrong_init…); `label_*`; pre-translated `template_en/hi/mr` |

| Table | Purpose | Key columns and constraints |
|---|---|---|
| **mutants** | Mutant fingerprints | FK `challenge_id`, `mistake_class_id`; `operator`; `source`; `fingerprint` bit string; index `challenge_id` |
| **glossary_terms** | Technical-term rendering | PK `term_en`; `term_hi`, `term_mr`; `keep_english` |

### Learning activity

| Table | Purpose | Key columns and constraints |
|---|---|---|
| **attempts** | Each submission | `id` PK; FK `user_id`, `challenge_id` (NULL for foreign); `foreign_problem_hash`; `code`; `verified`; `overall` (passed / failed); `hardcode_flag`; `large_paste_flag`; `duration_ms`; indexes by `(user_id, created_at)` and `(user_id, challenge_id)` |
| **test_results** | Outcome per test | FK `attempt_id` (cascade), `test_id` (NULL for fuzz); `outcome` in (passed, wrong_output, runtime_error, timeout); `is_fuzz`; `failure_class`; `runtime_ms` |
| **diagnoses** | Deterministic diagnosis | `attempt_id` UNIQUE; `mistake_class_id` (NULL = no known pattern); `confidence` 0–1; `divergence_json`; `failure_class`; `vector_id` |
| **concept_mastery** | Per-concept skill | PK `(user_id, concept_id)`; `score` default 1000; `attempts`; `updated_at` |
| **user_mistake_counts** | Recurring mistake counters | PK `(user_id, mistake_class_id)`; `count`; `last_seen_at` |
| **hint_state** | Hint unlock state | PK `(user_id, challenge_key)`; `max_level_unlocked` 1–4; `attempts_since_hint`; `first_attempt_at` |
| **hints_issued** | Audit of hints | `level` 1–4; `language`; `guard_passed`; `guard_retries`; `source` in (llm, template) |
| **explainbacks** | Spoken explanation scoring | `transcript`; `score` 0–1; `passed`; `language` |

### Conversation, voice and operations

| Table | Purpose | Key columns and constraints |
|---|---|---|
| **chat_threads** | Private threads | `id` UUID PK; FK `user_id`; `challenge_key` (NULL = general); `language`; `participant_type` (ai_mentor default; human_mentor reserved); `archived`; index `(user_id, updated_at DESC)` |
| **chat_messages** | Messages and guard audit | FK `thread_id` (cascade); `role` in (user, assistant); `content`; `input_mode` (text / voice); `detected_language`; `guard_passed`; `guard_retries`; `tool_calls_json`; index `(thread_id, id)` |
| **voice_usage** | Credit accounting | `kind` in (tts, stt); `characters_or_seconds`; `cache_hit`; index `created_at` for the daily cap |
| **tts_cache** | Audio cache index | PK `hash` = sha256(text\|voice\|lang\|model); `file_path`; `language`; `characters` |
| **redteam_runs** | Leak-rate evidence | `prompt`; `language`; `leaked`; `blocked_by`; `run_at` |

## 6.5 Data provenance

| Dataset | Source | Notes |
|---|---|---|
| **Challenges (8–10), reference solutions** | Authored by the team | Original, no scraping; references never sent to the client |

| Dataset | Source | Notes |
|---|---|---|
| **Visible / hidden tests** | Authored plus type-driven fuzz generators | Hidden tests never leave the server |
| **Mutants** | Generated from references via AST operators | Cached per challenge |
| **Labelled buggy submissions (~40)** | Authored with known mistake classes | Used for diagnosis accuracy |
| **Red-team prompts (~50)** | Authored in EN / HI / MR | “Just give me the answer” in many phrasings; used for hint-leak rate |
| **Sandbox attack snippets (6+)** | Authored | File read, socket, fork bomb, infinite loop, big allocation, import abuse |
| **Glossary and locale templates** | Authored; reviewed by Hindi and Marathi speakers on the team | Multilingual quality |
| **Third-party page content** | Text the user chose to analyse | Not stored; only a hash is kept |

## 6.6 Retention and privacy

- No accounts or emails; learners are identified by a locally generated UUID.
- Voice input is discarded after transcription; no user audio is stored. The TTS cache holds only guard-approved explanatory text.
- Profile, habits, chat and memory are viewable, editable and deletable; `memory_enabled` switches memory off.
- A privacy policy (`docs/privacy-policy.md`) states what is stored (code, profile, audio) and how to delete it.

# 7. API Architecture

## 7.1 Conventions

| | |
|---|---|
| **Base URL** | `http://localhost:8000` (default) or the hosted HTTPS URL set in the extension options |
| **Authentication** | Header `X-API-Token: <token>` on every endpoint except `/health`, plus `user_id` in the body or path. A user can only reach their own data; `user_id` must match the id bound to the install. Admin endpoints use an admin token. |
| **Contract-first** | `openapi.yaml` is written first (in the first 15 minutes of the build) so frontend and extension can work against mocks; the web client is typed from it. |
| **Streaming** | SSE (`text/event-stream`) for chat and optionally explanations |
| **Validation limits** | `code` ≤ 20,000 chars; `text` ≤ 2,000 chars; `problem_text` ≤ 8,000 chars; `examples` ≤ 10; `lang` ∈ {en, hi, mr}; `level` ∈ 1–4; audio ≤ 25 MB / 60 s |
| **Error body** | `{"error":{"code":"<CODE>","message":"<human text>","details":{...}}}` |

| Error code | HTTP | Meaning |
|---|---|---|
| `UNAUTHORIZED` | 401 | Missing or invalid token |
| `FORBIDDEN` | 403 | Accessing another user's data or thread |
| `HINT_LOCKED` | 403 | Requested hint tier not yet unlocked (returns unlock requirements) |
| `NOT_FOUND` | 404 | Unknown id |
| `VALIDATION_ERROR` | 422 | Body fails schema validation |
| `CODE_REJECTED` | 400 | AST policy rejected the learner code (reason included) |
| `UNSUPPORTED_LANGUAGE` | 400 | Language not in en / hi / mr |
| `CREDIT_CAP_REACHED` | 402 | Daily voice character cap hit; client should use browser TTS |
| `RATE_LIMITED` | 429 | Per-token limit exceeded |
| `UPSTREAM_FAILED` | 502 | LLM or ElevenLabs failed; response includes a `fallback` hint |

## 7.2 Endpoint catalogue

| # | Endpoint | Method | Auth | Purpose |
|---|---|---|---|---|
| 1 | `/health` | GET | None | Liveness: LLM, ElevenLabs and sandbox mode status |
| 2 | `/users` | POST | Token | Create a local user |
| 3 | `/users/{id}/settings` | GET / PUT | Token | Language, verbosity, style, voice |
| 4 | `/users/{id}/profile` | GET | Token | View learned patterns and habits |
| 5 | `/users/{id}/data` | DELETE | Token | Erase all user data |
| 6 | `/challenges` | GET | Token | List challenges (localised titles) |
| 7 | `/challenges/{id}` | GET | Token | Challenge detail and visible tests only |
| 8 | `/submissions` | POST | Token | Run code on visible, hidden and fuzz tests; return diagnosis |
| 9 | `/analyze` | POST | Token | Extension analysis on any problem (oracle, fuzz, trace, mutant match) |

| # | Endpoint | Method | Auth | Purpose |
|---|---|---|---|---|
| 10 | `/hint` | POST | Token | Tiered, guarded hint |
| 11 | `/explain` | POST | Token | Debug / explain / concept |
| 12 | `/chat/threads` | POST / GET | Token | Create / list chat threads |
| 13 | `/chat/threads/{tid}` | GET / DELETE | Token | Read / delete a thread |
| 14 | `/chat/threads/{tid}/messages` | POST | Token | Send a message; SSE reply |
| 15 | `/tts` | POST | Token | Approved text to audio (cached) |
| 16 | `/stt` | POST | Token | Audio to transcript |
| 17 | `/explainback` | POST | Token | Score a spoken explanation |
| 18 | `/dashboard/{user_id}` | GET | Token | Mastery, mistakes, recent diagnoses, credits |
| 19 | `/recommend/{user_id}` | GET | Token | Next challenge |
| 20 | `/i18n/detect` | POST | Token | Language detection incl. code-mixed flag |
| 21 | `/admin/redteam` | POST | Admin | Run the red-team set; return leak rate |
| 22 | `/admin/attacks` | POST | Admin | Run the sandbox attack set |

## 7.3 Key contracts

### POST /submissions

Request: `user_id`, `challenge_id`, `code`, `paste_chars`, `elapsed_ms`. Hidden-test results never include input or expected values.

```
{
  "attempt_id": 17,
  "overall": "failed",
  "results": [
    {"test":"v1","kind":"visible","outcome":"wrong_output",
     "input":"[1,2]","expected":"2","actual":"3"},
    {"test":"h1","kind":"hidden","outcome":"wrong_output",
     "failure_class":"empty input"}
  ],
  "hardcode_flag": false,
  "large_paste_flag": false,
  "explainback_required": false,
  "diagnosis": {
    "mistake_class": "off_by_one_bound", "confidence": 0.82,
    "divergence": {"iteration":3,"variable":"total",
                   "expected":7,"actual":5,"line":6}
  }
}
```

### POST /hint

Request: `user_id`, `challenge_key`, `code`, `level` (null = next allowed), `language`. Response: hint `level`, `text`, `language`, a `guard` report (`passed`, `retries`, `source`) and `next_unlock` (attempts and seconds still needed). Requesting a tier above the unlocked maximum returns `HINT_LOCKED`; an LLM outage returns a template hint.

### POST /chat/threads/{tid}/messages (SSE)

Request: `user_id`, `content` (1–2,000 chars), `input_mode`, optional `code_context` and `challenge_key`. The response stream emits `event: token` chunks and finishes with `event: done`

carrying `message_id`, the `guard` report, `detected_language` and the `tool_calls` used. On LLM failure the stream switches to a rule-based reply with `"mode":"rule_based"`.

### POST /analyze (extension)

Request: `user_id`, `problem_text`, `signature`, `code`, `examples`, `language`. Response includes `verified` (true / false), per-test outcomes, `failure_class`, `trace`, `mistake_class`, `confidence`, a localised `explanation`, and the language. If the oracle fails the visible examples, `verified` is false and no fuzzing is performed.

### POST /tts and POST /stt

`/tts` returns `audio/mpeg` with `X-Cache: hit|miss` and `X-Chars-Used` headers, or `{"fallback":"browser_tts"}` on failure; it only accepts text produced by approved endpoints. `/stt` accepts multipart `audio` (webm / mp3 / wav) and returns `transcript` plus `detected_language`.

### Admin evidence endpoints

`/admin/redteam` returns `{total, leaked, leak_rate, by_language}`; `/admin/attacks` returns `{total, blocked, details[{name, blocked, reason}]}`. Together with the diagnosis-accuracy test they provide the numbers shown to judges.

# 8. AI / ML Architecture

> **Design principle**
> Deterministic components decide; AI components word, translate and converse. If every AI call fails, the product still diagnoses, hints (templates) and explains in all three languages.

## 8.1 Where AI is used — and where it is not

| Task | AI used? | Method |
|---|---|---|
| **Running tests, outcomes** | No | Sandbox |
| **Bug localisation** | No | Divergence tracer |
| **Mistake classification** | No | Mutation fingerprint, AST checks |
| **Fuzzing, hardcode detection** | No | Generators, AST patterns |
| **Mastery update** | No | Elo / BKT |
| **Wording explanations and hints** | Yes (optional) | LLM with template fallback |
| **Oracle for foreign problems** | Yes | LLM, validated on visible examples |
| **Chat replies** | Yes (optional) | LLM agent with restricted tools; rule-based fallback |
| **Translation** | Yes (optional) | LLM + glossary; pre-translated templates fallback |
| **Explain-back scoring** | Yes | STT + LLM rubric; keyword rubric fallback |
| **Memory retrieval** | Yes (embeddings) | MiniLM + Chroma |

## 8.2 Component specification and fallbacks

| Component | Type | Input → Output | Fallback |
|---|---|---|---|
| **Oracle generator** | LLM | Problem text, signature → brute-force reference | Examples-only checks, label `unverified` |
| **Fuzzer / shrinker** | Algorithmic | Signature + two functions → minimal failing input, failure class | n/a |
| **Divergence tracer** | Algorithmic | Two traces → first differing step | Final-output / iteration-count comparison |
| **Mutation matcher** | Algorithmic (Hamming) | Pass/fail vector → mistake class + confidence | "No known pattern" below threshold |
| **Hint generator** | LLM + Guard | Diagnosis, tier, profile, language → hint text | Tiered template hints |
| **Hint Guard** | Algorithmic | Candidate text → approve / block | After 2 retries → template |
| **Mentor chat agent** | LLM with tools | Message, context → guarded reply | Rule-based intent chat |
| **Explain-back scorer** | STT + LLM rubric | Transcript, diagnosis rubric → score, feedback | Keyword rubric match |
| **Memory retrieval** | Embeddings + Chroma | New diagnosis → top-3 similar past diagnoses | Skip recall line |
| **Adaptive model** | Elo / BKT | Outcome, difficulty → concept scores | n/a |
| **Personalisation** | Algorithmic + prompt context | Code, behaviour events → profile summary | Default profile |

| Component | Type | Input → Output | Fallback |
|---|---|---|---|
| **Translation** | LLM + glossary | Neutral text → HI / MR text | Pre-translated templates |
| **Language detection** | Rules | Text → en / hi / mr, code-mixed flag | User setting |

## 8.3 LLM abstraction

All LLM traffic goes through a single entry point, `llm.complete()` in `llm/client.py`, which hides the provider (Gemini free tier or Groq, with local Ollama as fallback). Prompt templates for explain, hint, chat, rubric and translate live in `llm/prompts/`. Each prompt receives structured, language-neutral facts plus the profile summary; none receives reference code.

## 8.4 Evaluation framework

| Metric | Test file / dataset | Target / report |
|---|---|---|
| **Diagnosis accuracy** | `test_diagnosis_labeled.py`, ~40 labelled submissions | Per-class accuracy; "no known pattern" rate |
| **Hint-leak rate** | `test_guard_redteam.py`, ~50 prompts in EN / HI / MR incl. via chat | Target 0; report honestly with failures listed |
| **Chat guard** | `test_chat_guard.py` | Chat cannot emit unguarded solutions |
| **Attack-block rate** | `test_sandbox_attacks.py`, 6+ attacks | 100% |
| **Language detection** | `test_language_detect.py`, ~30 samples incl. code-mixed | Accuracy |
| **Multilingual quality** | ~20 reviewed outputs | Reviewer scores 1–5 |
| **Personalisation sanity** | Same failure with two profiles | Wording differs; outcomes and tiers identical |

Results are written into `docs/evidence/` (`diagnosis_accuracy.md`, `hint_leak_report.md`, `sandbox_attacks.md`, `multilingual_eval.md`) and shown during the demo.

# 9. Technology Stack

## 9.1 Backend

| Layer | Technology | Purpose |
|---|---|---|
| **Language** | Python 3.11 | Backend, sandbox harness, tracer |
| **Framework** | FastAPI + Uvicorn | REST API and SSE streaming |
| **Validation** | Pydantic | Request / response validation |
| **ORM** | SQLAlchemy | Database access |
| **Database** | SQLite | Attempts, mastery, profile, chat, caches |
| **Vector store** | Chroma | Per-user diagnosis memory |
| **Embeddings** | `all-MiniLM-L6-v2` (sentence-transformers) | Local embeddings; no paid API |
| **LLM** | Gemini free tier or Groq; Ollama local fallback | Wording, oracle, chat, translation, rubric scoring |
| **Voice** | ElevenLabs TTS + Scribe | Audio output and spoken explain-back |
| **Language detection** | Script check + stop-word heuristic (optional `langdetect`) | Detect en / hi / mr |
| **Container** | Docker + Compose | Packaging and sandbox hardening |

## 9.2 Frontend, extension and DevOps

| Area | Technology | Purpose |
|---|---|---|
| **Web app** | React + Vite + TypeScript; Tailwind CSS | UI and styling |
| **Editor** | Monaco Editor | Code editing and paste-event capture |
| **Charts** | Recharts | Skill radar, mistake heatmap |
| **Web i18n** | i18next + react-i18next | UI in en / hi / mr |
| **Extension platform** | Chrome Manifest V3; `@crxjs/vite-plugin` | Runtime and bundling with HMR |
| **Extension UI** | React (panel, popup); Shadow DOM (toolbar) | Isolated styling from host pages |
| **Extension audio** | Offscreen document | Service workers cannot play audio |
| **Local run** | `docker compose up` | One-command start |
| **Hosted backup** | AWS EC2 (or tunnel) + Caddy | HTTPS fallback backend |
| **Distribution** | Zip of `dist/` on a GitHub Release (load-unpacked) | No Chrome Web Store fee or review delay |

## 9.3 Key technologies explained

| Technology | What it is | How CodeQuest uses it | Caveat |
|---|---|---|---|
| `sys.settrace` | Python hook called on every line, call and return | Records (line, locals) for reference and learner to find the first divergence | Slows execution (step cap); needs overlapping variable names |
| **AST mutation** | Small deliberate edits to parsed code | Creates mutants that mimic beginner bugs and yield fingerprints | Only covers mutation classes we define |

| Technology | What it is | How CodeQuest uses it | Caveat |
|---|---|---|---|
| **Differential fuzzing + shrinking** | Random inputs compared across two implementations, then minimised | Finds and simplifies counterexamples | Needs a trustworthy reference (oracle verification) |
| **Hint Guard** | Execution-based leak detection | Runs and AST-compares hint code; blocks solutions | Threshold tuned on the red-team set |
| **MV3 service worker + offscreen doc** | Short-lived background worker plus a DOM context for audio | All backend calls and audio playback | Playback needs a user gesture |
| **Shadow DOM** | Encapsulated DOM subtree | Toolbar immune to host-page CSS | — |
| **Chroma + MiniLM** | Vector store and local embedding model | “Same mistake as before” recall | Wording only; model baked into the image |
| **Elo / BKT** | Skill estimation from outcomes | Concept mastery weighted by difficulty | Needs enough attempts to stabilise |
| **SSE** | One-way HTTP streaming | Token-by-token chat and explanations | Multi-instance scaling needs sticky sessions or pub/sub |

# 10. Security Architecture

## 10.1 Trust boundaries

Three boundaries drive the security design: (1) learner code and third-party page content are untrusted; (2) clients (browser, extension) are untrusted with respect to secrets and scoring; (3) AI outputs are untrusted until the guard approves them.

## 10.2 Threat model

| Threat | Mitigation |
|---|---|
| **Malicious learner code (file read, network, fork bomb, infinite loop, memory)** | AST allowlist + `python -I` + empty env + temp cwd + rlimits + wall timeout + process-group kill; Docker `--network none --memory 256m --cpus 1 --pids-limit 64`; non-root, read-only FS with tmpfs |
| **Remote code execution via the hosted backend** | Per-token auth and rate limits; no egress from containers; hosted tier treated as a hostile-input service |
| **Hidden test or reference leakage** | Never serialised to the client; hidden results show only a failure class; fuzz seeds per attempt |
| **Hint / solution leakage (hints, chat, voice, translation)** | Hint Guard on every output path; re-run after translation; TTS only for approved text |
| **Prompt injection via page content or tool output** | Treated as data; fixed system rules; restricted tools; guard on outputs |
| **API key exposure** | Keys only in backend `.env`; extension holds no keys; content script never calls APIs directly |
| **Credit drain (ElevenLabs)** | Daily character cap per token, cache, rate limit |
| **Cross-user data access in chat / profile** | Every query scoped by `user_id`; thread ownership check → `FORBIDDEN` |
| **CORS abuse** | Allowlist `chrome-extension://<fixed id>` and the web origin only |
| **Privacy of code and voice** | Voice discarded after transcription; export / delete everything; documented privacy policy |
| **Dependency compromise** | Pinned versions; no remote code in the extension (MV3 compliant) |
| **Misuse on live contests** | Extension shows a disabled state on contest pages |
| **Voice identity confusion** | No voice authentication is claimed; explain-back is a comprehension check, not identity proof |

## 10.3 Data protection

| Data | Stored where | Protection |
|---|---|---|
| **Learner code** | SQLite | Local volume; deletable |
| **Profile, habits, chat** | SQLite + Chroma | Scoped by user id; deletable; `memory_enabled` switch |
| **Voice audio** | Not stored | Discarded after speech-to-text |
| **TTS audio** | Disk cache | Contains only approved explanatory text |
| **Tokens** | `api_tokens` | Hash only; revocable |
| **Secrets** | `.env` | Not committed; `.env.example` only |

## 10.4 Extension security

- Permissions: `activeTab`, `contextMenus`, `storage`, `offscreen`, content-script matches (`<all_urls>` in the demo build for “any tab”), and `host_permissions` for the backend URL.
- UI is isolated in Shadow DOM; all code is bundled (no remotely hosted code).
- A fixed manifest `key` gives every install the same extension ID, making the CORS allowlist reliable.
- The extension reads only text the user selects and the visible problem statement; no bulk scraping of editorials or hidden tests.

# 11. Deployment Architecture

## 11.1 Two deployment tiers

| Tier | Purpose | Notes |
|---|---|---|
| **Primary: local** | Judges and the demo machine | `docker compose up`; backend at `localhost:8000`; no external dependency for the core demo |
| **Backup: hosted** | When someone's Docker fails | EC2 + Caddy HTTPS, or a tunnel as emergency; extension options page switches the backend URL |

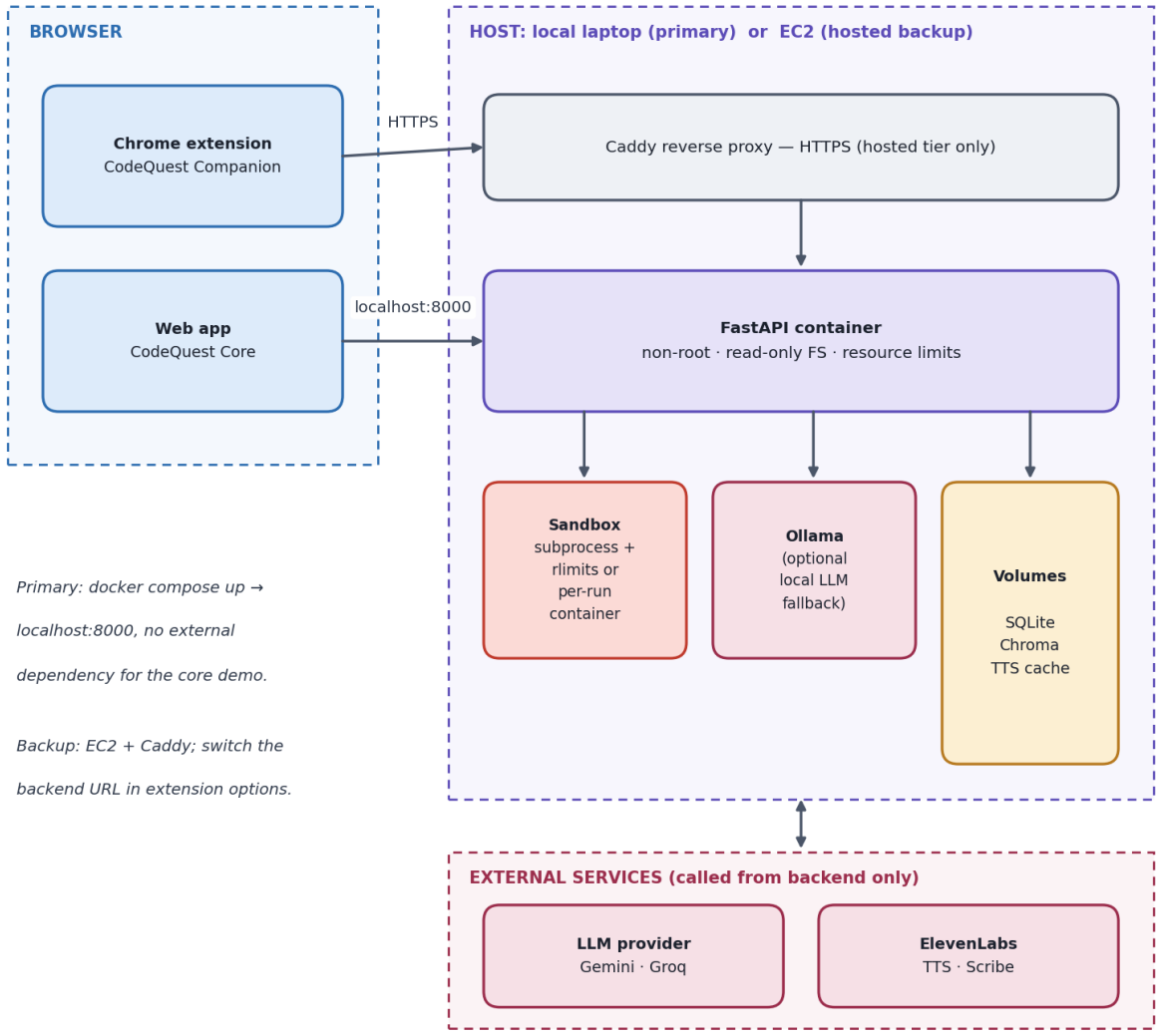



*Figure 8. Deployment architecture (local primary, hosted backup)*

## 11.2 Packaging

| Item | Detail |
|---|---|
| **Backend image** | Python slim, pinned dependencies, MiniLM model baked in at build, non-root user |
| **Compose** | Volumes for SQLite, Chroma and the TTS cache; resource limits; no network for sandbox runs |
| **Extension** | `vite build` → zip the contents of `dist/` (manifest at zip root) → GitHub Release `codequest-extension.zip` |
| **Install (3 steps)** | Download and unzip → `chrome://extensions` → Developer mode → Load unpacked |

| Item | Detail |
|---|---|
| **Chrome Web Store** | Not used (avoids the registration fee and review delay) |
| **Fixed extension ID** | `key` in `manifest.json` |
| **Offline kit** | Zip + `docker save` image on USB; 3-minute backup demo video |

## 11.3 Hosted-backup hardening

Shared token per install; per-token rate limits and a daily ElevenLabs character cap; no outbound network from the sandbox; non-root containers with a read-only filesystem and tmpfs; billing alerts; instance stopped after the event. EC2 free-tier or credit terms depend on account creation date `[VERIFY on aws.amazon.com/free]`.

## 11.4 Cost

| Item | Cost |
|---|---|
| **Extension distribution (GitHub zip)** | Free |
| **ElevenLabs** | Covered by the organiser offer `[VERIFY]` |
| **LLM** | Free tiers or local Ollama `[VERIFY limits]` |
| **Chroma, SQLite, MiniLM** | Free |
| **Hosted backup** | Credits / free tier `[VERIFY]`; optional domain ≈ $1–10 |

## 11.5 Walking skeleton and demo-day checklist

The first two hours deliver a walking skeleton — `/health`, Dockerfile, compose, extension options page (backend URL), CORS with the fixed extension ID, and one `/tts` call from the toolbar — to surface CORS, permission and audio-playback problems early.

1. Run the full demo on a fresh Chrome profile.
2. Pre-render demo audio (EN / HI / MR) into the cache.
3. Keep the browser `speechSynthesis` fallback enabled.
4. Record a 3-minute backup video.
5. Warm the hosted backend before the slot.
6. Bring the zip and `docker save` image on USB.
7. Keep Ollama ready if the machine can handle it.
8. Keep a Hindi and a Marathi demo scenario ready.